

This document provides the full UC security proof that are omitted from the main paper due to space constraints. All materials are provided in a compact but complete form to ensure the soundness and completeness of the UC framework.

SCOPE AND STRUCTURE

This document includes: (i) the ledger functionality $\mathcal{L}_{MC}$; (ii) the ledger functionality $\mathcal{L}_{CMC}$; (iii) the simulator $\mathcal{S}_{CMC}$; and (iii) auxiliary notes linking the simulator to the proof sketch in the main paper.

Public Observability and Interaction:

As ledger functionalities, both $\mathcal{L}_{MC}$ and $\mathcal{L}_{CMC}$ are publicly observable. Any party, including the adversary $\mathcal{A}$ (and the simulator $\mathcal{S}_{CMC}$ in the ideal execution), can query their public state at any time. Furthermore, the environment $\mathcal{Z}$ has direct read access to these ledgers, enforcing strict consistency between the on-chain state and the protocol execution. All Broadcast messages emitted by the ledgers are immediately delivered to the adversary/simulator.

PAYMENT CHANNEL LEDGER FUNCTIONALITY $\mathcal{L}_{MC}$

The ledger functionality $\mathcal{L}_{MC}$ interacts with channel participants U , Supervisory Users SU , the cross-channel ledger $\mathcal{L}_{CMC}$, and the ideal functionality $\mathcal{F}_{CMC}$. It maintains on-chain states and collaterals for channels indexed by num^{θ_r} .

User joining. Upon receiving $(join, pid, num^{\theta_r}, amount)$ from U , lock amount from U 's on-chain balance into channel num^{θ_r} . Record U as an active member of channel num^{θ_r} . Broadcast $(joined, pid, num^{\theta_r}, U, amount)$.

Supervisory User election. Upon receiving $(register_su, pid, num^{\theta_r}, deposit)$ from U , lock deposit and record U as an active SU^{θ_r} for channel num^{θ_r} . Broadcast $(su_elected, pid, num^{\theta_r}, U)$.

Channel state update. Upon receiving $(update, pid, num^{\theta_r}, balances)$ validly signed by all active SUs of num^{θ_r} , update the recorded balances of all channel members according to balances. Release the locked deposit to the respective SUs . Broadcast $(state_updated, pid, num^{\theta_r})$.

Timeout withdrawal. Upon receiving $(timeout_withdraw, pid, num^{\theta_r})$ from U after the expiration time without a valid state update, release U 's balance according to the latest finalized state. Confiscate the locked deposit of all active SUs in num^{θ_r} . Broadcast $(slashed, pid, num^{\theta_r}, SUs)$.

Dispute and penalty. Upon receiving $(dispute, pid, num^{\theta_r}, evidence)$ from U , or upon receiving the cross-layer directive $(slash_su, num^{\theta_r}, SU)$ from $\mathcal{L}_{CMC}$, verify the cryptographic evidence or the directive's origin. If valid, confiscate the specific SU 's deposit and allocate it to the honest members. Broadcast $(slashed, pid, num^{\theta_r}, SU)$.

User exit. Upon receiving $(exit, pid, num^{\theta_r})$ from U with an allocated balance $amount'$, release $amount'$ to U 's on-chain address and remove U from num^{θ_r} . Broadcast $(exited, pid, num^{\theta_r}, U)$.

Fig. 1: Payment Channel Ledger functionality $\mathcal{L}_{MC}$

CROSS CHANNEL LEDGER FUNCTIONALITY $\mathcal{L}_{CMC}$

The ledger functionality $\mathcal{L}_{CMC}$ interacts with Supervisory Users SU , Watchtowers WT , protocol participants, the ideal functionality $\mathcal{F}_{CMC}$, and the underlying $\mathcal{L}_{MC}$ instances. It acts as the global registry and cross-channel arbitrator.

Watchtower registration. Upon receiving $(register_wt, id_{WT}, deposit)$ from a Watchtower, lock deposit and record id_{WT} as an active global WT . Broadcast $(wt_registered, id_{WT})$.

Watchtower deregistration. Upon receiving $(deregister_wt, id_{WT})$ from an active WT , verify the absence of pending cross-channel disputes involving id_{WT} . If verified, release deposit to id_{WT} and revoke its active status. Broadcast $(wt_deregistered, id_{WT})$.

Global Watchtower slashing. Upon receiving $(slash_wt, id_{WT}, fraud_proof)$ from any party, verify $fraud_proof$. If valid, confiscate the global deposit of id_{WT} , allocate compensations, and broadcast $(wt_slashed, id_{WT})$.

Cross-channel Supervisory User slashing. Upon receiving $(cross_dispute, pid, num^{\theta_r}, SU, evidence)$ from any party, verify the cross-channel evidence against SU . If valid, send the slashing directive $(slash_su, num^{\theta_r}, SU)$ to the specific $\mathcal{L}_{MC}$ corresponding to num^{θ_r} . Broadcast $(cross_slashed, pid, SU)$.

Fig. 2: Cross Channel Ledger functionality $\mathcal{L}_{CMC}$

SIMULATION $\mathcal{S}_{CMC}$

The simulator $\mathcal{S}_{CMC}$ interacts with the adversary $\mathcal{A}$ and the environment $\mathcal{Z}$ while accessing $\mathcal{F}_{CMC}$, $\mathcal{L}_{MC}$, $\mathcal{L}_{CMC}$, and $\mathcal{S}_{CMC}$. The ledger is honest, whereas users, supervisory users, and the Watchtower WT may be statically corrupted.

Internal state. For each payment identifier pid , the simulator stores a tuple (NUM, ID, m, t) and a deviation log $Dev[pid]$.

Stage I: Payment Request.

- *Case I1 (honest payer, honest $SU_a^{\theta_0}$).*
Upon receiving $(leak_pay, pid, pr, pe, m, t, NUM, ID)$ from $\mathcal{F}_{CMC}$, deliver (pay, pid, pr, pe, m, t) to $\mathcal{A}$ on behalf of $pr_j^{\theta_0}$. Send (pay_ok, pid) to $\mathcal{F}_{CMC}$ on behalf of $SU_a^{\theta_0}$.
- *Case I2 (corrupt payer, honest $SU_a^{\theta_0}$).*

Extract $(\text{pay}, \text{pid}, \text{pr}, \text{pe}, m, t)$ sent by $\mathcal{A}$ to $SU_a^{\theta_0}$. If valid, send $(\text{pay}, \text{pid}, \text{pr}, \text{pe}, m, t)$ to $\mathcal{F}_{CMC}$ on behalf of $pr_j^{\theta_0}$. Upon receiving $(\text{leak_pay}, \text{pid}, \text{pr}, \text{pe}, m, t, \text{NUM}, \text{ID})$ from $\mathcal{F}_{CMC}$, send $(\text{pay_ok}, \text{pid})$ to $\mathcal{F}_{CMC}$ on behalf of $SU_a^{\theta_0}$. Otherwise, send $(\text{abort}, \text{pid})$ to $\mathcal{F}_{CMC}$.

- *Case I3 (honest payer, corrupt $SU_a^{\theta_0}$).*

Upon receiving $(\text{leak_pay}, \text{pid}, \text{pr}, \text{pe}, m, t, \text{NUM}, \text{ID})$ from $\mathcal{F}_{CMC}$, deliver $(\text{pay}, \text{pid}, \text{pr}, \text{pe}, m, t)$ to $\mathcal{A}$ on behalf of $pr_j^{\theta_0}$. If $\mathcal{A}$ accepts and successfully processes the request, send $(\text{pay_ok}, \text{pid})$ to $\mathcal{F}_{CMC}$. Upon $\mathcal{A}$ refusing the request or observing a timeout, send $(\text{abort}, \text{pid})$ to $\mathcal{F}_{CMC}$.

- *Case I4 (corrupt payer, corrupt $SU_a^{\theta_0}$).*

Extract $(\text{pay}, \text{pid}, \text{pr}, \text{pe}, m, t)$ exchanged by $\mathcal{A}$. If $\mathcal{A}$ successfully processes a valid request, send $(\text{pay}, \text{pid}, \text{pr}, \text{pe}, m, t)$ to $\mathcal{F}_{CMC}$ on behalf of the corrupted $pr_j^{\theta_0}$. Upon receiving $(\text{leak_pay}, \text{pid}, \text{pr}, \text{pe}, m, t, \text{NUM}, \text{ID})$ from $\mathcal{F}_{CMC}$, send $(\text{pay_ok}, \text{pid})$ to $\mathcal{F}_{CMC}$ on behalf of the corrupted $SU_a^{\theta_0}$. Otherwise, send $(\text{abort}, \text{pid})$ to $\mathcal{F}_{CMC}$.

Stage II: Fund Holding.

- *Case H1 (honest supervisory users).*

Upon receiving $(\text{leak_hold_req}, \text{pid}, \text{num}^{\theta_r}, \text{ID}_r, m)$ from $\mathcal{F}_{CMC}$, deliver $(\text{hold_req}, \text{pid}, \text{num}^{\theta_r}, \text{ID}_r, m)$ to $\mathcal{A}$ on behalf of $SU_a^{\theta_0}$. Deliver $(\text{sync_req}, \text{pid}, \text{ID}_r, m)$ to $\mathcal{A}$ on behalf of $SU_a^{\theta_r}$. Send $(\text{hold_ok}, \text{pid}, \text{num}^{\theta_r})$ to $\mathcal{F}_{CMC}$ and $\mathcal{A}$ on behalf of $SU_b^{\theta_r}$.

- *Case H2 (corrupt supervisory users).*

Extract $(\text{hold_req}, \text{pid}, \text{num}^{\theta_r}, \text{ID}_r, m)$ and $(\text{sync_req}, \text{pid}, \text{ID}_r, m)$ exchanged by $\mathcal{A}$. If $\mathcal{A}$ successfully holds the funds and outputs a valid $(\text{hold_ok}, \text{pid}, \text{num}^{\theta_r})$ for all channels, send $(\text{hold_ok}, \text{pid}, \text{num}^{\theta_r})$ to $\mathcal{F}_{CMC}$ on behalf of $SU_b^{\theta_r}$. Otherwise, send $(\text{stop_hold}, \text{pid})$ to $\mathcal{F}_{CMC}$.

- *Case H3 (honest $SU_a^{\theta_0}$, mixed corruptions of other supervisory users).*

Upon receiving $(\text{leak_hold_req}, \text{pid}, \text{num}^{\theta_r}, \text{ID}_r, m)$ from $\mathcal{F}_{CMC}$, deliver $(\text{hold_req}, \text{pid}, \text{num}^{\theta_r}, \text{ID}_r, m)$ to $\mathcal{A}$ on behalf of $SU_a^{\theta_0}$. Extract $(\text{hold_ok}, \text{pid}, \text{num}^{\theta_r})$ sent by $\mathcal{A}$ to $SU_a^{\theta_0}$. Upon extracting an invalid message or observing a timeout from any corrupted channel, send $(\text{stop_hold}, \text{pid})$ to $\mathcal{F}_{CMC}$.

- *Case H4 (corrupt $SU_a^{\theta_0}$, mixed corruptions of other supervisory users).*

Let E be the first honest entity encountering an invalid input. If E is $SU_a^{\theta_r}$, extract $(\text{hold_req}, \text{pid}, \text{num}^{\theta_r}, \text{ID}_r, m)$ from $\mathcal{A}$. If E is $SU_b^{\theta_r}$, extract $(\text{sync_req}, \text{pid}, \text{ID}_r, m)$ from $\mathcal{A}$. Upon extracting an invalid message or observing a timeout, send $(\text{stop_hold}, \text{pid})$ to $\mathcal{F}_{CMC}$. Otherwise, send $(\text{hold_ok}, \text{pid}, \text{num}^{\theta_r})$ to $\mathcal{F}_{CMC}$.

Stage III: Watchtower Assistance.

- **Honest WT, Honest $SU_a^{\theta_0}$.**

- *Case W1 (All $SU_b^{\theta_r}$ honest):* Upon receiving $(\text{leak_match_eval}, \text{pid}, \text{outcome})$ from $\mathcal{F}_{CMC}$, deliver $(\text{reqwt}, \text{pid}, \text{NUM}, C(m), \text{pr } f_k(\text{ID}), CQ, E)$ and $(\text{wt_update}, \text{pid}, \text{num}^{\theta_r}, \text{UpLT}^{\theta_r})$ for all $\text{num}^{\theta_r} \in \text{NUM}$ to $\mathcal{A}$.
- *Case W2 (All $SU_b^{\theta_r}$ corrupt):* Extract $(\text{wt_update}, \text{pid}, \text{num}^{\theta_r}, \text{UpLT}^{\theta_r})$ sent by $\mathcal{A}$ for all $\text{num}^{\theta_r} \in \text{NUM}$. If valid, send them to $\mathcal{F}_{CMC}$. Upon receiving $(\text{leak_match_eval}, \text{pid}, \text{outcome})$ from $\mathcal{F}_{CMC}$, deliver $(\text{reqwt}, \text{pid}, \text{NUM}, C(m), \text{pr } f_k(\text{ID}), CQ, E)$ to $\mathcal{A}$. If any update is invalid, send $(\text{force_abort}, \text{pid})$ to $\mathcal{F}_{CMC}$.
- *Case W3 (Mixed corruptions of $SU_b^{\theta_r}$):* Extract $(\text{wt_update}, \text{pid}, \text{num}^{\theta_r}, \text{UpLT}^{\theta_r})$ sent by $\mathcal{A}$ for corrupted channels. If valid, send them to $\mathcal{F}_{CMC}$. Upon receiving $(\text{leak_match_eval}, \text{pid}, \text{outcome})$, deliver $(\text{reqwt}, \text{pid}, \text{NUM}, C(m), \text{pr } f_k(\text{ID}), CQ, E)$ and the honest $(\text{wt_update}, \text{pid}, \text{num}^{\theta_r}, \text{UpLT}^{\theta_r})$ to $\mathcal{A}$. If any corrupted update is invalid, send $(\text{force_abort}, \text{pid})$ to $\mathcal{F}_{CMC}$.

- **Honest WT, Corrupt $SU_a^{\theta_0}$.**

- *Case W4 (All $SU_b^{\theta_r}$ honest):* Extract $(\text{reqwt}, \text{pid}, \text{NUM}, C(m), \text{pr } f_k(\text{ID}), CQ, E)$ sent by $\mathcal{A}$ to the honest WT. If valid, send it to $\mathcal{F}_{CMC}$. Upon receiving $(\text{leak_match_eval}, \text{pid}, \text{outcome})$, deliver $(\text{wt_update}, \text{pid}, \text{num}^{\theta_r}, \text{UpLT}^{\theta_r})$ for all $\text{num}^{\theta_r} \in \text{NUM}$ to $\mathcal{A}$. If the request is invalid, send $(\text{force_abort}, \text{pid})$ to $\mathcal{F}_{CMC}$.
- *Case W5 (All $SU_b^{\theta_r}$ corrupt):* Extract $(\text{reqwt}, \text{pid}, \text{NUM}, C(m), \text{pr } f_k(\text{ID}), CQ, E)$ and $(\text{wt_update}, \text{pid}, \text{num}^{\theta_r}, \text{UpLT}^{\theta_r})$ for all $\text{num}^{\theta_r} \in \text{NUM}$ generated internally by $\mathcal{A}$. If all are valid, send them to $\mathcal{F}_{CMC}$. Upon receiving $(\text{leak_match_eval}, \text{pid}, \text{outcome})$ from $\mathcal{F}_{CMC}$, ignore. If any input is invalid, send $(\text{force_abort}, \text{pid})$ to $\mathcal{F}_{CMC}$.
- *Case W6 (Mixed corruptions of $SU_b^{\theta_r}$):* Extract $(\text{reqwt}, \text{pid}, \text{NUM}, C(m), \text{pr } f_k(\text{ID}), CQ, E)$ and $(\text{wt_update}, \text{pid}, \text{num}^{\theta_r}, \text{UpLT}^{\theta_r})$ for the corrupted channels from $\mathcal{A}$. If valid, send them to $\mathcal{F}_{CMC}$. Upon receiving $(\text{leak_match_eval}, \text{pid}, \text{outcome})$ from $\mathcal{F}_{CMC}$, deliver the honest $(\text{wt_update}, \text{pid}, \text{num}^{\theta_r}, \text{UpLT}^{\theta_r})$ to $\mathcal{A}$. If invalid, send $(\text{force_abort}, \text{pid})$ to $\mathcal{F}_{CMC}$.

- **Corrupt WT, Honest $SU_a^{\theta_0}$.**

- *Case W7 (All $SU_b^{\theta_r}$ honest):* Deliver $(\text{reqwt}, \text{pid}, \text{NUM}, C(m), \text{pr } f_k(\text{ID}), CQ, E)$ and $(\text{wt_update}, \text{pid}, \text{num}^{\theta_r}, \text{UpLT}^{\theta_r})$ for all $\text{num}^{\theta_r} \in \text{NUM}$ to $\mathcal{A}$. Upon receiving $(\text{leak_match_eval}, \text{pid}, \text{outcome})$ from $\mathcal{F}_{CMC}$, record outcome.

- Case W8 (All $SU_b^{\theta_r}$ corrupt): Deliver $(reqwt, pid, NUM, C(m), prf_k(ID), CQ, E)$ to $\mathcal{A}$. Extract $(wt_update, pid, num^{\theta_r}, UpLT^{\theta_r})$ routed by $\mathcal{A}$ to the corrupted WT. If valid, send them to $\mathcal{F}_{CMC}$ and record outcome. If invalid, send $(force_abort, pid)$ to $\mathcal{F}_{CMC}$.
- Case W9 (Mixed corruptions of $SU_b^{\theta_r}$): Deliver $(reqwt, pid, NUM, C(m), prf_k(ID), CQ, E)$ and the honest $(wt_update, pid, num^{\theta_r}, UpLT^{\theta_r})$ to $\mathcal{A}$. Extract the corrupted $(wt_update, pid, num^{\theta_r}, UpLT^{\theta_r})$ from $\mathcal{A}$. If valid, send them to $\mathcal{F}_{CMC}$ and record outcome. If invalid, send $(force_abort, pid)$ to $\mathcal{F}_{CMC}$.
- **Corrupt WT, Corrupt $SU_a^{\theta_0}$.**
 - Case W10 (All $SU_b^{\theta_r}$ honest): Deliver all honest $(wt_update, pid, num^{\theta_r}, UpLT^{\theta_r})$ to $\mathcal{A}$. Extract $(reqwt, pid, NUM, C(m), prf_k(ID), CQ, E)$ generated internally by $\mathcal{A}$. If valid, send it to $\mathcal{F}_{CMC}$ and record outcome. If invalid, send $(force_abort, pid)$ to $\mathcal{F}_{CMC}$.
 - Case W11 (All $SU_b^{\theta_r}$ corrupt): Extract all internally routed $(reqwt, pid, NUM, C(m), prf_k(ID), CQ, E)$ and $(wt_update, pid, num^{\theta_r}, UpLT^{\theta_r})$ from $\mathcal{A}$. If valid, send them to $\mathcal{F}_{CMC}$ and record outcome. If invalid, send $(force_abort, pid)$ to $\mathcal{F}_{CMC}$.
 - Case W12 (Mixed corruptions of $SU_b^{\theta_r}$): Deliver the honest $(wt_update, pid, num^{\theta_r}, UpLT^{\theta_r})$ to $\mathcal{A}$. Extract $(reqwt, pid, NUM, C(m), prf_k(ID), CQ, E)$ and the corrupted $(wt_update, pid, num^{\theta_r}, UpLT^{\theta_r})$ from $\mathcal{A}$. If valid, send them to $\mathcal{F}_{CMC}$ and record outcome. If invalid, send $(force_abort, pid)$ to $\mathcal{F}_{CMC}$.

Stage IV: Supervision of the Watchtower.

- **Honest WT.**
 - Case S1 (Both $SU_a^{\theta_r}, SU_b^{\theta_r}$ honest): Generate the evaluation response $resp_{WT}$ containing the recorded outcome and Δ_r . Deliver $resp_{WT}$ to $\mathcal{A}$ on behalf of WT. Deliver the intermediate synchronization message to $\mathcal{A}$ on behalf of $SU_a^{\theta_r}$. Execute the local off-chain state update on behalf of the SUs. Commit the final settlement to $\mathcal{L}_{CMC}$ on behalf of WT. Send $(eval_ok, pid)$ to $\mathcal{F}_{CMC}$.
 - Case S2 (Corrupt $SU_a^{\theta_r}$, Honest $SU_b^{\theta_r}$): Generate and deliver $resp_{WT}$ to $\mathcal{A}$ on behalf of WT. Extract the intermediate synchronization message sent by $\mathcal{A}$ to $SU_b^{\theta_r}$. If strictly matching the recorded outcome and Δ_r , execute the local off-chain state update on behalf of $SU_b^{\theta_r}$, commit the final settlement to $\mathcal{L}_{CMC}$ on behalf of WT, and send $(eval_ok, pid)$ to $\mathcal{F}_{CMC}$. If invalid or dropped, send $(force_abort, pid)$ to $\mathcal{F}_{CMC}$ and invoke timeout withdrawal on $\mathcal{L}_{MC}$.
 - Case S3 (Honest $SU_a^{\theta_r}$, Corrupt $SU_b^{\theta_r}$): Generate and deliver $resp_{WT}$ to $\mathcal{A}$ on behalf of WT. Deliver the intermediate synchronization message to $\mathcal{A}$ on behalf of $SU_a^{\theta_r}$. Extract the local off-chain state confirmation from $\mathcal{A}$. If valid, commit the final settlement to $\mathcal{L}_{CMC}$ on behalf of WT and send $(eval_ok, pid)$ to $\mathcal{F}_{CMC}$. Otherwise, send $(force_abort, pid)$ to $\mathcal{F}_{CMC}$ and invoke timeout withdrawal on $\mathcal{L}_{MC}$.
 - Case S4 (Both $SU_a^{\theta_r}, SU_b^{\theta_r}$ corrupt): Generate and deliver $resp_{WT}$ to $\mathcal{A}$ on behalf of WT. Extract the local off-chain state confirmation from $\mathcal{A}$. If $\mathcal{A}$ validly confirms the state matching Δ_r , commit the final settlement to $\mathcal{L}_{CMC}$ on behalf of WT and send $(eval_ok, pid)$ to $\mathcal{F}_{CMC}$. Otherwise, send $(force_abort, pid)$ to $\mathcal{F}_{CMC}$.
- **Corrupt Watchtower.**
 - Case S5 (Both $SU_a^{\theta_r}, SU_b^{\theta_r}$ honest): Extract $resp_{WT}$ sent by $\mathcal{A}$ to $SU_a^{\theta_r}$. If strictly matching outcome and Δ_r , deliver the intermediate message to $\mathcal{A}$ on behalf of $SU_a^{\theta_r}$, execute the local off-chain state update, and extract the final settlement commitment on $\mathcal{L}_{CMC}$ from $\mathcal{A}$. Send $(eval_ok, pid)$ to $\mathcal{F}_{CMC}$. If fraudulent or contradictory, construct $fraud_proof$, send $(fraud, pid, fraud_proof)$ to $\mathcal{F}_{CMC}$, and invoke $slash_wt$ on $\mathcal{L}_{CMC}$ on behalf of the SUs. If formatted invalidly, send $(force_abort, pid)$.
 - Case S6 (Corrupt $SU_a^{\theta_r}$, Honest $SU_b^{\theta_r}$): Extract the intermediate synchronization message sent by $\mathcal{A}$ to $SU_b^{\theta_r}$, or extract any $slash_wt$ invocation on $\mathcal{L}_{CMC}$. If $\mathcal{A}$ submits a valid $fraud_proof$ to $\mathcal{L}_{CMC}$, send $(fraud, pid, fraud_proof)$ to $\mathcal{F}_{CMC}$. If $\mathcal{A}$ forwards a valid intermediate message matching Δ_r , execute the local off-chain state update on behalf of $SU_b^{\theta_r}$, extract the final settlement commitment on $\mathcal{L}_{CMC}$ from $\mathcal{A}$, and send $(eval_ok, pid)$ to $\mathcal{F}_{CMC}$. Otherwise, send $(force_abort, pid)$.
 - Case S7 (Honest $SU_a^{\theta_r}$, Corrupt $SU_b^{\theta_r}$): Extract $resp_{WT}$ sent by $\mathcal{A}$ to $SU_a^{\theta_r}$. If fraudulent, construct $fraud_proof$, send $(fraud, pid, fraud_proof)$ to $\mathcal{F}_{CMC}$, and invoke $slash_wt$ on $\mathcal{L}_{CMC}$ on behalf of $SU_a^{\theta_r}$. If valid, deliver the intermediate message to $\mathcal{A}$ on behalf of $SU_a^{\theta_r}$. Extract the local off-chain state confirmation and the final settlement commitment on $\mathcal{L}_{CMC}$ from $\mathcal{A}$. If valid, send $(eval_ok, pid)$. Otherwise, send $(force_abort, pid)$.
 - Case S8 (Both $SU_a^{\theta_r}, SU_b^{\theta_r}$ corrupt): Extract all local off-chain updates and global $\mathcal{L}_{CMC}$ interactions from $\mathcal{A}$. If $\mathcal{A}$ successfully invokes $slash_wt$ on $\mathcal{L}_{CMC}$ using a valid $fraud_proof$, send $(fraud, pid, fraud_proof)$ to $\mathcal{F}_{CMC}$. If $\mathcal{A}$ correctly confirms the local state and successfully commits the settlement to $\mathcal{L}_{CMC}$ matching the recorded outcome, send $(eval_ok, pid)$ to $\mathcal{F}_{CMC}$. Otherwise, send $(force_abort, pid)$ to $\mathcal{F}_{CMC}$.

Fig. 3: Simulation $\mathcal{S}_{CMC}$

MAPPING TO THE SECURITY PROOF

This section clarifies how the specific stages of the simulator $\mathcal{S}_{CMC}$ and the underlying ledger functionalities ($\mathcal{L}_{MC}, \mathcal{L}_{CMC}$) correspond to the formal security arguments presented in Section V of the main paper.

- **Stage I: Payment Request.** The forwarding of payment requests and the distinction between in-channel and cross-channel transactions in the proof sketch correspond precisely to this initial setup stage of $\mathcal{S}_{CMC}$.
- **Stage II: Fund Holding.** The tentative fund locking and pre-deduction arguments in the proof sketch are realized by Stage II, which leverages the simulated multi-party payment channels $\mathcal{L}_{MC}$ to ensure that concurrent or delayed cross-channel executions cannot reuse the same balance.
- **Stage III: Watchtower Assistance.** The arguments regarding the cryptographic condition evaluation, input extraction, and the Watchtower's prerequisite updates are strictly mapped to the evaluation mechanisms defined in Stage III of $\mathcal{S}_{CMC}$.
- **Stage IV: Supervision of the Watchtower.** This terminal stage comprehensively absorbs the remaining proof arguments regarding protocol finality and adversarial misbehavior:
 - *Successful Completion:* The double-layer synchronization chain ($WT \rightarrow SU_a \rightarrow SU_b$) and the final balance updates are modeled in the honest execution branches of Stage IV, where all users strictly execute local off-chain state updates and commit the final settlement to $\mathcal{L}_{CMC}$.
 - *Accountability and Slashing:* Any cryptographic fraud, adversarial compliance, or protocol stalling by a corrupted WT or SU directly triggers the construction of `fraud_proof`, the invocation of the `slash_wt` penalty on $\mathcal{L}_{CMC}$, or timeout withdrawals on $\mathcal{L}_{MC}$, matching the accountability claims in the proof sketch.

Overall, each step of the proof sketch has a direct, explicitly modeled counterpart in the simulator state machine and ledger specifications provided in this document.

LEAKAGE AND ASSUMPTIONS

This section summarizes the information leakage and modeling assumptions implicitly captured by the ideal functionalities and simulators.

- a) *Permitted leakage.*: The adversary may learn the following information: (i) the public payment identifier pid , (ii) the set of channels involved in a cross-channel payment, (iii) whether a payment is successfully completed or aborted, and (iv) public ledger events, including deposit locking, release, and slashing.
- b) *Protected information.*: Plaintext user identities, channel balances, and intermediate routing information are not revealed. The watchtower observes only encrypted identifiers and encrypted balance updates, which are computationally indistinguishable from random values under the assumed security of the Paillier cryptosystem.
- c) *Corruption assumptions.*: The model assumes static corruption. Users, channel supervisory users, and the watchtower may be corrupted, while the ledger functionalities are incorruptible.
- d) *Network assumptions.*: Communication channels are authenticated. The adversary controls message scheduling and delivery but cannot forge messages from honest parties.

COMPOSABILITY NOTES

The security definition and proof are given in the Universal Composability framework. By the UC composition theorem, the proposed cross-channel payment protocol remains secure under arbitrary concurrent executions and composition with other UC-secure protocols.

In particular, the protocol can be safely composed with: (i) multiple independent payment channels, (ii) multiple concurrent cross-channel payments, and (iii) other blockchain applications that interact with the same ledger functionalities.

The separation between protocol execution, watchtower assistance, and ledger adjudication ensures that composition does not introduce additional trust assumptions or leakage beyond what is explicitly modeled in the ideal functionalities.